ServiX

Gestión de clientes, servicios y atención con IA

MANUAL TÉCNICO

Manual Técnico de ServiX

Guía para desarrollar, configurar, probar, integrar y mantener la aplicación Android ServiX

Proyecto	ServiX — Gestión de clientes, servicios y atención con IA
Código del documento	DOC-05
Versión	1.0
Fecha	01/11/2026
Estándar de referencia	Buenas prácticas de documentación técnica (ISO/IEC/IEEE 26514)
Institución	Universidad de San Buenaventura
Asignatura	Optativa II
Docente	Ing. Jairo Armando Salcedo Aranda

Equipo de trabajo

Nicolas Quiroz Gomez	Product Owner
Juan Sebastian Garzon Suarez	Scrum Master
Juan Esteban Olaya Muñoz	Developer

Control del documento

Información general

Título	Manual Técnico de ServiX
Código	DOC-05
Versión	1.0
Fecha de emisión	01/11/2026
Estado	Versión 1.0 emitida; aprobaciones pendientes
Proyecto	ServiX
Autor	Equipo ServiX
Clasificación	Uso académico; sin información confidencial

Historial de revisiones

Versión	Fecha	Descripción del cambio	Autor
0.9	01/11/2026	Borrador inicial en formato plano (archivado como respaldo v1).	Equipo ServiX
1.0	01/11/2026	Rediseño completo: portada, control de versiones, numeración, figuras y tablas rotuladas, mayor profundidad.	Equipo ServiX

Aprobaciones

Rol	Nombre	Fecha	Firma / estado
Elaboró	Juan Esteban Olaya Muñoz (Developer)	Pendiente	Pendiente
Revisó	Juan Sebastian Garzon Suarez (Scrum Master)	Pendiente	Pendiente
Aprobó	Nicolas Quiroz Gomez (Product Owner)	Pendiente	Pendiente

Las aprobaciones se encuentran **pendientes**: no se registran firmas ni fechas de aprobación hasta que los responsables las emitan.

Tabla de contenido

1 Introducción **5**
2 Entorno de desarrollo **6**
2.1 Recomendaciones para equipos con poca memoria 6
3 Estructura del proyecto **7**
4 Arquitectura resumida **8**
4.1 Interfaz de la aplicación 9
5 Dependencias y versiones **10**
6 Configuración de Firebase **11**
6.1 Crear y vincular el proyecto 11
6.2 Desplegar las reglas de Firestore 11
7 Inteligencia artificial **12**
8 Integración con Make **13**
8.1 Configuración en local.properties 13
8.2 Configuración con variable de entorno 13
9 Variables, secretos y archivos que no se versionan **14**
10 Compilación y ejecución **15**
10.1 Compilar el APK de depuración 15
10.2 Instalar y ejecutar 15
10.3 Compilar una versión release 15
11 Pruebas **16**
12 Integración continua **17**
13 Solución de errores frecuentes **18**
14 Mantenimiento **19**
15 Convenciones de trabajo **20**

Lista de figuras

Figura 1. Arquitectura por capas (resumen) . . . 8
Figura 2. Componentes principales y sus dependencias . . . 8
Figura 3. Pantallas del administrador: inicio, clientes, servicios y chat . . . 9
Figura 4. Flujo de integración continua . . . 17

Lista de tablas

Tabla 1. Requisitos del entorno de desarrollo . . . 6
Tabla 2. Directorios principales . . . 7
Tabla 3. Referencia rápida de comandos . . . 9
Tabla 4. Dependencias principales y versiones . . . 10
Tabla 5. Configuración del módulo . . . 10
Tabla 6. Configuración de la IA . . . 12
Tabla 7. Eventos enviados a Make . . . 13
Tabla 8. Secretos y archivos locales . . . 14
Tabla 9. Clases de prueba . . . 16
Tabla 10. Pasos del pipeline . . . 17
Tabla 11. Errores frecuentes y soluciones . . . 18
Tabla 12. Convenciones de ramas, commits y Pull Requests . . . 20

1 Introducción

Este manual explica cómo preparar el entorno, compilar, ejecutar, probar y mantener ServiX. Está dirigido a quien deba trabajar en el código: el equipo actual o futuros colaboradores. Complementa al Manual de Instalación (DOC-06), que se centra en poner la aplicación en un teléfono, y a los documentos de arquitectura (DOC-02), Firebase (DOC-07) e IA (DOC-08).

> **Advertencia. Secretos**
> Este manual no contiene claves, tokens ni URL de webhook. Donde se necesita un valor secreto se indica un marcador como `<URL_DEL_WEBHOOK>` que cada persona debe sustituir con su propio valor y **no versionar**.

2 Entorno de desarrollo

Tabla 1. Requisitos del entorno de desarrollo

Componente	Versión o requisito	Notas
Sistema operativo	Windows 10 u 11, Linux o macOS	Equipo de referencia: Windows 10 con 4 GB de RAM.
JDK	21 (recomendado)	El SDK 36 exige Java 21 para las pruebas Robolectric; con 17 la compilación puede funcionar pero las pruebas de interfaz fallan.
Android SDK	Plataforma `android-36.1` y build-tools 36.0.0	Android Studio los instala al sincronizar.
Gradle	9.3.1 (wrapper)	Use siempre `gradlew`; no instale Gradle por separado.
Android Gradle Plugin	9.1.1	Definido en `gradle/libs.versions.toml`.
Kotlin	2.2.10	Con el plugin de Compose.
Git	2.x	Para clonar y trabajar con ramas.
Firebase CLI (opcional)	Última estable	Para desplegar `firestore.rules`.
IDE	Android Studio o Visual Studio Code	Android Studio es más pesado; en 4 GB de RAM conviene usar línea de comandos.

2.1 Recomendaciones para equipos con poca memoria

- Cierre navegadores y otras aplicaciones antes de compilar; el proceso de Gradle usa hasta 3 GB (`org.gradle.jvmargs=-Xmx3072m`).
- Use `--no-daemon` si el sistema se vuelve inestable y evite abrir Android Studio y un emulador a la vez.
- Apóyese en Azure Pipelines para compilar y probar cuando el equipo local sea insuficiente: el run exitoso tardó 9 min 42 s frente a más de 20 min en local.
- No hay virtualización disponible en el equipo de referencia, por lo que no se usa emulador; se prueba en un dispositivo físico.

3 Estructura del proyecto

El repositorio contiene un único módulo Gradle (`app`) y documentación en `docs/`. La siguiente tabla y el árbol describen su organización real.

Árbol del repositorio (resumido)

```
ServiX/
├── app/
│   ├── build.gradle.kts            # configuración del módulo (SDK, dependencias,
  BuildConfig)
│   ├── google-services.json        # configuración del cliente Firebase
│   └── src/
│       ├── main/java/com/example/
│       │   ├── app/                # MainActivity, ServiXApplication
│       │   ├── core/               # di, navigation, theme, utils
│       │   ├── data/               # ai, firebase (+dto), mock, network
│       │   ├── domain/             # model, repository, usecase
│       │   ├── presentation/       # components, dialogs, screens, viewmodel
│       │   └── ui/screens/         # AuthScreen
│       ├── main/res/               # strings, colores, temas, íconos
│       ├── test/java/com/example/  # pruebas JUnit y Robolectric
│       └── androidTest/            # prueba instrumentada de ejemplo
├── gradle/
│   ├── libs.versions.toml          # catálogo de versiones
│   └── wrapper/                    # Gradle Wrapper
├── docs/                           # ARQUITECTURA, FIREBASE, CHAT_IA, PRUEBAS, AUDITORIA,
  AZURE_DEVOPS
├── azure-pipelines.yml             # pipeline ServiX-CI
├── firestore.rules                 # reglas de seguridad de Firestore
├── firebase.json / .firebaserc     # configuración de la Firebase CLI
├── build.gradle.kts, settings.gradle.kts, gradle.properties
├── .env.example                    # documenta que no se requieren variables obligatorias
├── CONTRIBUTING.md, README.md
└── gradlew, gradlew.bat
```

Tabla 2. Directorios principales

Ruta	Contenido
`app/src/main/java/com/example/domain`	Modelos, interfaces de repositorio y casos de uso.
`app/src/main/java/com/example/data`	Repositorios Firebase, Gemini, Make y simulados.
`app/src/main/java/com/example/presentation`	Pantallas, componentes, diálogos y `MainViewModel`.
`app/src/main/java/com/example/core`	Contenedor de dependencias, rutas, tema y utilidades.
`app/src/test`	Nueve clases de prueba (JUnit y Robolectric).
`docs`	Documentación Markdown del repositorio.

4 Arquitectura resumida

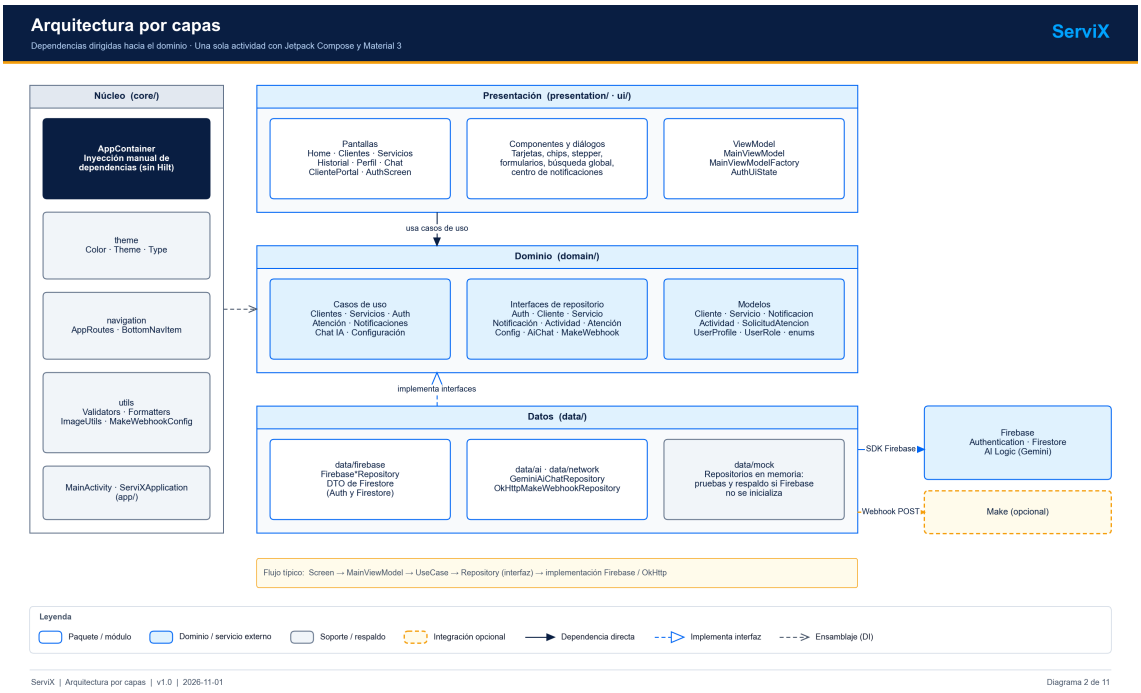


Figura 1. Arquitectura por capas (resumen)

Fuente: Diagrama del Equipo ServiX (04_Arquitectura_Diagramas).

La aplicación sigue el flujo **UI → ViewModel → UseCase → Repository → fuente de datos**. El dominio no depende de Android ni de Firebase. `AppContainer` compone las dependencias a mano. El detalle de las decisiones se encuentra en DOC-02.

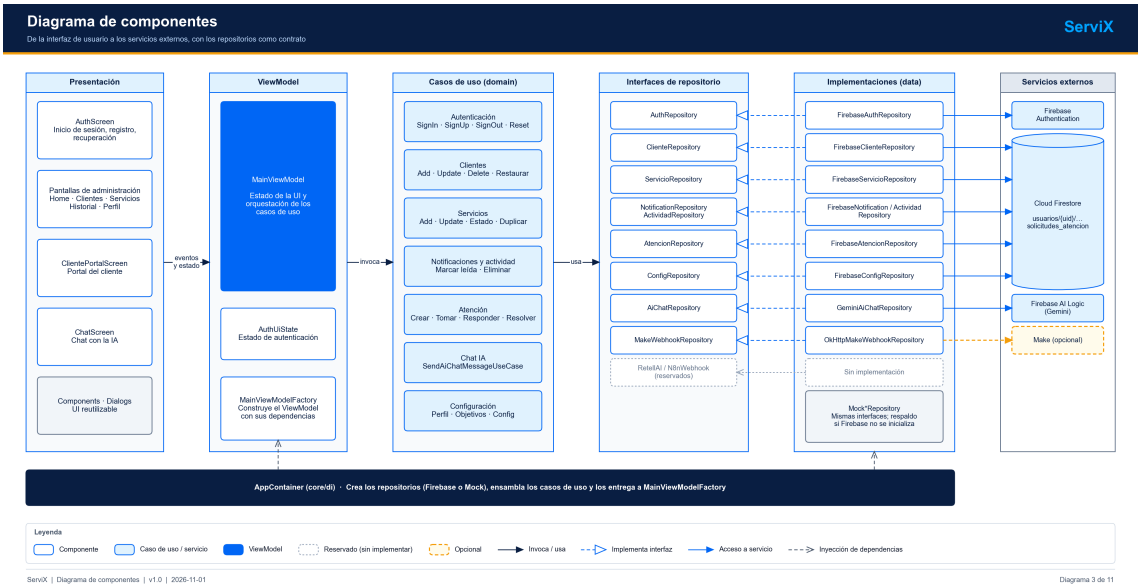


Figura 2. Componentes principales y sus dependencias

Fuente: Diagrama del Equipo ServiX (04_Arquitectura_Diagramas).

4.1 Interfaz de la aplicación

Las siguientes figuras son renders de Robolectric con datos de demostración; permiten ubicar las pantallas que corresponden a cada paquete de `presentation`.

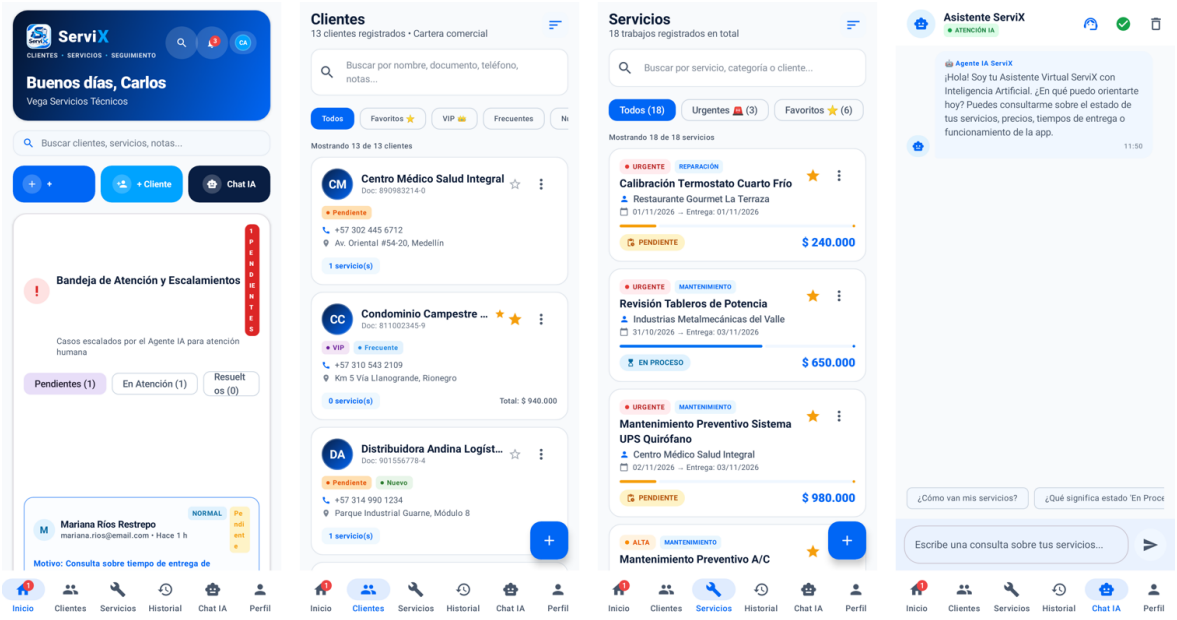


Figura 3. Pantallas del administrador: inicio, clientes, servicios y chat

Fuente: Render de Robolectric con datos de demostración (09_Evidencias/Capturas_App).

Tabla 3. Referencia rápida de comandos

Tarea	Comando
Compilar APK de depuración	`gradlew.bat assembleDebug`
Pruebas unitarias y Robolectric	`gradlew.bat testDebugUnitTest`
Android Lint (no ejecutado)	`gradlew.bat lintDebug`
Pruebas instrumentadas (no ejecutadas)	`gradlew.bat connectedDebugAndroidTest`
Instalar el APK	`adb install -r app/build/outputs/apk/debug/app-debug.apk`
Desplegar reglas de Firestore	`firebase deploy --only firestore:rules`

5 Dependencias y versiones

Las versiones se declaran en `gradle/libs.versions.toml`. La tabla recoge las que usa el módulo `app` (las demás entradas del catálogo, como Room, Moshi o Coil, están comentadas o sin usar).

Tabla 4. Dependencias principales y versiones

Dependencia	Versión	Uso
Android Gradle Plugin	9.1.1	Compilación Android.
Kotlin y plugin de Compose	2.2.10	Lenguaje y compilador de Compose.
Compose BOM	2024.09.00	Material 3, UI, gráficos, iconos extendidos.
Firebase BoM	34.17.0	Auth, Firestore, AI Logic y App Check (reCAPTCHA).
Google Services (plugin)	4.5.0	Procesa `google-services.json`.
Navigation Compose	2.8.9	Navegación entre pantallas.
Lifecycle (runtime y ViewModel Compose)	2.8.7	Estado y ciclo de vida.
Activity Compose	1.10.1	Integración con `ComponentActivity`.
Core KTX	1.18.0	Extensiones de Kotlin para Android.
Corrutinas	1.10.2	Asincronía.
OkHttp y logging-interceptor	4.10.0	Cliente HTTP del webhook de Make.
Retrofit	2.12.0	Declarada como dependencia.
JUnit	4.13.2	Pruebas unitarias.
Robolectric	4.16.1	Pruebas con Android en JVM.
Roborazzi	1.59.0	Capturas de pantalla de Compose.
AndroidX Test (core, junit, runner), Espresso	1.6.1, 1.3.0, 1.6.2, 3.7.0	Pruebas de interfaz.

Tabla 5. Configuración del módulo

Parámetro	Valor
`applicationId`	`com.example.servix`
`namespace`	`com.example`
`minSdk` / `targetSdk` / `compileSdk`	26 / 36 / 36 (minorApiLevel 1)
`versionCode` / `versionName`	1 / 1.0
Compatibilidad Java	11 (código fuente y destino)
Funciones habilitadas	Compose y BuildConfig

6 Configuración de Firebase

ServiX usa el proyecto Firebase `servix-453fb`. El archivo `app/google-services.json` conecta la aplicación con ese proyecto; el repositorio ya lo incluye. Si el equipo necesita trabajar con otro proyecto, siga estos pasos.

6.1 Crear y vincular el proyecto

1. En la consola de Firebase cree un proyecto (o abra el existente).
2. Añada una aplicación Android con el identificador `com.example.servix`.
3. Descargue `google-services.json` y colóquelo en `app/`.
4. En **Authentication → Método de acceso**, habilite *Correo electrónico y contraseña*.
5. En **Firestore Database**, cree la base de datos en modo producción (las reglas se despliegan a continuación).
6. En **Firebase AI Logic**, habilite el proveedor *Gemini Developer API* para el proyecto.

6.2 Desplegar las reglas de Firestore

Despliegue de reglas con la Firebase CLI

```
npm install -g firebase-tools
firebase login
firebase deploy --only firestore:rules
```

`firebase.json` apunta a `firestore.rules` y `.firebaserc` fija el proyecto predeterminado. Tras el despliegue, compruebe la fecha de publicación en la pestaña *Reglas* de la consola. El detalle de cada regla está en DOC-07.

> **Advertencia. Restringir la clave de API**
> La clave del archivo `google-services.json` identifica al proyecto, no es un secreto de servidor, pero debe restringirse por paquete y huella SHA-1 en Google Cloud Console para evitar abuso (riesgo R-06, pendiente).

7 Inteligencia artificial

El chat usa Firebase AI Logic. No se necesita ninguna clave en el código, en `.env` ni en `local.properties`. El flujo y los parámetros son:

Tabla 6. Configuración de la IA

Elemento	Valor
Clase	`GeminiAiChatRepository`
Backend	`GenerativeBackend.googleAI()` (Gemini Developer API)
Modelo	`gemini-2.5-flash`
Temperatura / máximo de tokens	0,7 / 800
Historial	Últimos 6 mensajes (iniciando por un turno de usuario)
Contexto	Instrucción de sistema construida en `SendAiChatMessageUseCase.buildSystemContext`

Para verificar la IA: instale el APK, inicie sesión, abra **Chat IA** y envíe una consulta general. Si el chat responde con el mensaje de inconveniente temporal, revise que AI Logic esté habilitado, la conectividad y las cuotas. Esta comunicación real **no se ha verificado todavía** (casos CP-038 y CP-048).

8 Integración con Make

El envío de eventos a Make es opcional. La URL del webhook no se versiona; se define al compilar.

8.1 Configuración en local.properties

Archivo local.properties

```
# local.properties (no se versiona)
make.webhook.url=<URL_DEL_WEBHOOK>
```

8.2 Configuración con variable de entorno

Variable MAKE_WEBHOOK_URL

```
# Windows (PowerShell)
$env:MAKE_WEBHOOK_URL = "<URL_DEL_WEBHOOK>"
# Linux / macOS
export MAKE_WEBHOOK_URL="<URL_DEL_WEBHOOK>"
```

`app/build.gradle.kts` lee primero `make.webhook.url` de `local.properties` y, si no existe, la variable `MAKE_WEBHOOK_URL`; el valor se guarda en `BuildConfig.MAKE_WEBHOOK_URL`. `MakeWebhookConfig.isConfigured` indica si hay URL. Sin ella, los eventos se omiten y la aplicación funciona con normalidad. Después de cambiar la URL hay que recompilar.

Tabla 7. Eventos enviados a Make

Evento	Cuándo se envía
`SERVICIO_CREADO`	Al crear un servicio.
`SERVICIO_ACTUALIZADO`	Al actualizar un servicio.
`SERVICIO_COMPLETADO`	Al pasar un servicio al estado Completado.
Reporte manual y prueba de conexión	Desde Perfil, con respuesta sincrónica (código HTTP).

> **Nota. La URL queda en el APK**
> Aunque no está en el repositorio, el valor se compila dentro del APK y puede extraerse de él. Si el escenario de Make debe protegerse, la solución es enviar el evento desde un backend (riesgo R-04).

9 Variables, secretos y archivos que no se versionan

Tabla 8. Secretos y archivos locales

Elemento	¿Se versiona?	Dónde vive	Observación
`local.properties`	No	Cada equipo	Contiene `sdk.dir` y, opcionalmente, `make.webhook.url`.
`.env`	No	Cada equipo	Ignorado por `.gitignore`; ServiX no lo necesita para compilar.
`*.jks`, `*.keystore`	No	Cada equipo	Claves de firma de *release*.
`KEYSTORE_PATH`, `STORE_PASSWORD`, `KEY_PASSWORD`	No	Variables de entorno	Solo para firmar un *release*.
`MAKE_WEBHOOK_URL`	No	Variable de entorno o `local.properties`	URL del webhook.
`google-services.json`	Sí	Repositorio	Identificadores del proyecto Firebase (no secretos de servidor); restringir la clave.
`firestore.rules`, `firebase.json`	Sí	Repositorio	Reglas y configuración de la CLI.

10 Compilación y ejecución

10.1 Compilar el APK de depuración

Compilación

```
# Windows
gradlew.bat assembleDebug
# Linux / macOS
./gradlew assembleDebug
```

El APK queda en `app/build/outputs/apk/debug/app-debug.apk`. La primera compilación descarga dependencias y puede tardar más de 20 minutos en un equipo de 4 GB.

10.2 Instalar y ejecutar

Instalación por adb

```
adb devices
adb install -r app/build/outputs/apk/debug/app-debug.apk
```

También puede abrir el proyecto en Android Studio y pulsar *Run* con un dispositivo conectado.

10.3 Compilar una versión release

La configuración `release` exige las variables `KEYSTORE_PATH`, `STORE_PASSWORD` y `KEY_PASSWORD` y un almacén de claves con alias `upload`. No hay *release* firmado en el proyecto y el pipeline solo publica el APK de depuración.

11 Pruebas

Ejecución de las pruebas unitarias y Robolectric

```
gradlew.bat testDebugUnitTest
```

Tabla 9. Clases de prueba

Clase	Pruebas	Contenido
`ExampleUnitTest`	1	Prueba plantilla.
`ExampleRobolectricTest`	2	Recurso de cadena y lanzamiento de `MainActivity`.
`GreetingScreenshotTest`	1	Captura de `ServiXApp` con Roborazzi.
`ServixUnitTests`	5	Clientes, servicios y eliminación en cascada.
`ServixRepositoryTest`	3	Repositorios simulados.
`ServixAuthUnitTests`	5	Inicio y cierre de sesión y registro.
`ServixAtencionUnitTests`	4	Solicitudes de atención.
`ServixMakeIntegrationTest`	3	Eventos a Make con un doble.
`ServixRobolectricTest`	5	Navegación entre pantallas.

Resultados: 29 pruebas aprobadas el 31/10/2026 (local) y el 01/11/2026 (Azure Pipelines, run 20261101.3). Los informes quedan en `app/build/test-results/testDebugUnitTest/`. Android Lint y las pruebas instrumentadas no se han ejecutado.

> **Nota. Prueba adicional de capturas**
> El repositorio local contiene `ScreenshotsDocTest`, una prueba auxiliar de capturas para documentación que no forma parte de las 29 pruebas contabilizadas en este manual.

12 Integración continua

El pipeline `ServiX-CI` está definido en `azure-pipelines.yml`. Se dispara con cambios en `main` y con Pull Requests hacia `main`.

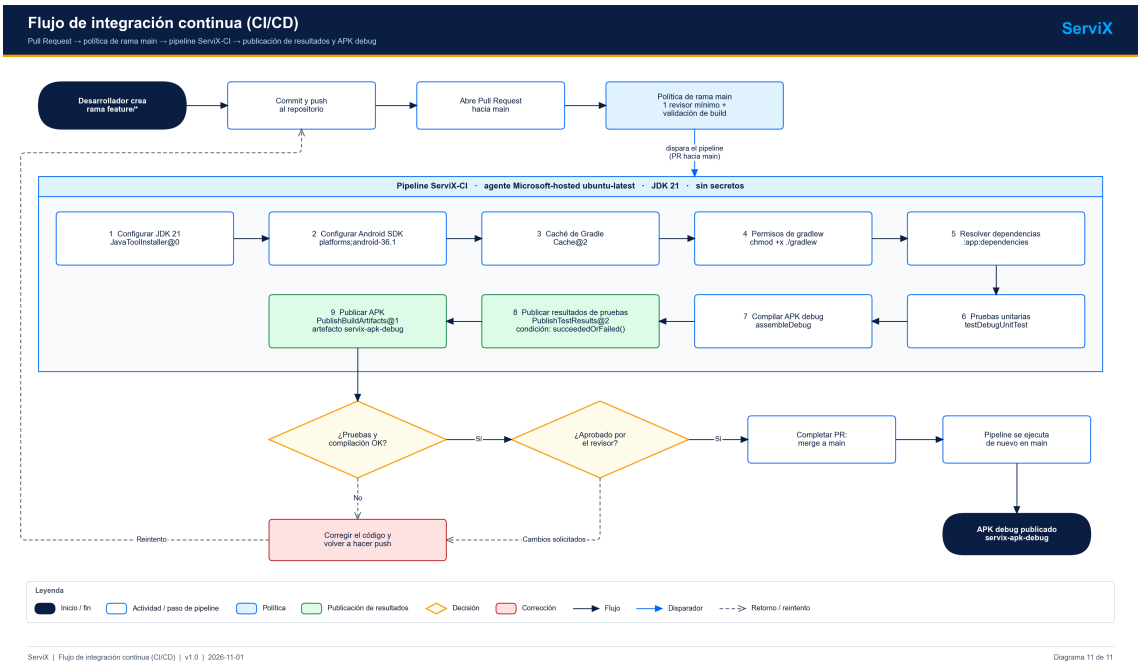


Figura 4. Flujo de integración continua

Fuente: Diagrama del Equipo ServiX (04_Arquitectura_Diagramas).

Tabla 10. Pasos del pipeline

Paso	Acción
Configurar JDK 21	`JavaToolInstaller@0` con versión 21.
Configurar Android SDK	Escribe `sdk.dir`, acepta licencias e instala `platforms;android-36.1` y `build-tools;36.0.0`.
Caché de Gradle	`Cache@2` con clave basada en los `*.gradle.kts`, el catálogo y el wrapper.
Resolver dependencias	`./gradlew --no-daemon :app:dependencies --configuration debugRuntimeClasspath`.
Pruebas unitarias	`./gradlew --no-daemon testDebugUnitTest`.
Compilar APK debug	`./gradlew --no-daemon assembleDebug`.
Publicar resultados	`PublishTestResults@2` (JUnit) con condición `succeededOrFailed()`.
Publicar APK	`PublishBuildArtifacts@1` con el artefacto `servix-apk-debug`.

13 Solución de errores frecuentes

Tabla 11. Errores frecuentes y soluciones

Síntoma	Causa	Solución
`gradlew` no se reconoce o falta `gradle-wrapper.jar`.	Wrapper ausente (ocurrió en el repositorio original, hallazgo H-06).	Restaurar `gradlew`, `gradlew.bat` y `gradle/wrapper/` desde el repositorio; en Linux ejecutar `chmod +x gradlew`.
Fallan las pruebas Robolectric o el compilador con mensajes de versión de Java.	El SDK 36 requiere Java 21; se usa JDK 17.	Instalar JDK 21 y apuntar `JAVA_HOME`. En Azure, usar `JavaToolInstaller` con versión 21 (run 2).
`Unresolved reference: util` en `build.gradle.kts`.	Se usa `java.util.Properties` sin importarlo.	Agregar `import java.util.Properties` al inicio del archivo.
El paso *Resolver dependencias* falla en el pipeline.	El comando se ejecuta en el proyecto raíz sin la configuración `debugRuntimeClasspath` (run 1).	Usar `:app:dependencies --configuration debugRuntimeClasspath`.
`OutOfMemoryError` o el equipo se congela.	4 GB de RAM insuficientes para Gradle y el IDE.	Cerrar aplicaciones, usar `--no-daemon`, reducir `org.gradle.jvmargs` o compilar en Azure Pipelines.
`SDK location not found`.	Falta `sdk.dir` en `local.properties`.	Abrir el proyecto en Android Studio o escribir `sdk.dir=<ruta-del-SDK>` en `local.properties`.
`google-services.json` no encontrado (advertencia).	Archivo ausente en `app/`.	Descargarlo de la consola de Firebase (la configuración usa la estrategia WARN).
El chat responde con el mensaje de inconveniente temporal.	AI Logic no habilitado, sin red o sin cuota.	Revisar la consola de Firebase, la conectividad y las cuotas.
Los eventos no llegan a Make.	URL no configurada o desactualizada.	Definir la URL, recompilar y usar la prueba de conexión del Perfil.
Aviso `ubuntu-latest will migrate` en el pipeline.	Microsoft migra la imagen `ubuntu-latest`.	Fijar una versión de imagen estable (riesgo R-12).

14 Mantenimiento

- Mantener actualizada la documentación de `docs/` y los documentos DOC-xx cuando cambie el comportamiento.
- Revisar mensualmente las advertencias del pipeline y las versiones del catálogo; no cambiar versiones sin evaluar compatibilidad.
- Probar con el wrapper antes de subir el cambio (`testDebugUnitTest` y `assembleDebug`).
- Revisar las reglas de Firestore ante cada nueva colección y volver a desplegarlas.
- Vigilar costos y cuotas de Gemini en la consola de Firebase.
- Atender la deuda técnica registrada: división de `MainViewModel`, aviso ante degradación a mock, rol de administrador asignado desde servidor, App Check.

15 Convenciones de trabajo

Tabla 12. Convenciones de ramas, commits y Pull Requests

Elemento	Convención
Ramas	`main` (estable), `develop` (integración), `feature/<descripcion-corta>` por funcionalidad o corrección.
Commits	Mensajes descriptivos con prefijo de tipo: `feat`, `fix`, `docs`, `chore`, `refactor`, `security`, `ci`; por ejemplo `fix(ci): usa JDK 21`.
Pull Requests	Hacia `main`, con política de un revisor; título claro, descripción del cambio y elemento de trabajo vinculado.
Antes del PR	Ejecutar `./gradlew testDebugUnitTest assembleDebug` y comprobar que no hay secretos.
Tras integrar	Actualizar el elemento en Azure Boards y confirmar que el pipeline de `main` pasa.
Registro de cambios	Añadir las entradas a `REPORTE_CAMBIOS_SERVIX.txt` sin modificar las anteriores.